

```

struct employee
{
    char name[30];
    int age;
    float bs;
};
typedef struct employee EMP;
EMP e1, e2;

```

Thus, by reducing the length and apparent complexity of data types, **typedef** can help to clarify source listing and save time and energy spent in understanding a program.

The above **typedef** can also be written as

```

typedef struct employee
{
    char name[30];
    int age;
    float bs;
} EMP;
EMP e1, e2;

```

typedef can also be used to rename pointer data types as shown below:

```

struct employee
{
    char name[30];
    int age;
    float bs;
}
typedef struct employee * PEMP;
PEMP p;
p->age = 32;

```

Typecasting

Sometimes we are required to force the compiler to explicitly convert the value of an expression to a particular data type. This would be clear from the following example:

```

#include <stdio.h>
void main()
{
    float a;
    int x = 6, y = 4;
    a = x / y;
    printf ( "\nValue of a = %f", a );
}

```

And here is the output...

Value of a = 1.000000

The answer turns out to be 1.000000 and not 1.5. This is because, 6 and 4 are both integers and hence $6 / 4$ yields an integer, 1. This 1 when stored in **a** is converted to 1.000000. But what if we don't want the quotient to be truncated? One solution is to make either **x** or **y** as **float**. Let us say that other requirements of the program do not permit us to do this. In such a case what do we do? Use type casting. The following program illustrates this.

```

#include <stdio.h>
void main()
{
    float a;
    int x = 6, y = 4;
    a = ( float ) x / y;
    printf ( "\nValue of a = %f", a );
}

```

And here is the output...